Kai 9000: A Complete Workflow Guide for an Open-Hardware Developer on a Galaxy A15 with Termux

TL;DR

- **Kai 9000 is a free, open-source (Apache-2.0), ad-free AI assistant by developer Simon Schubert** that connects to a large set of LLM providers (plus a no-key Free tier), runs on Android/iOS/desktop/web, and is distinguished by three power features directly useful to a hardware tinkerer: a built-in **Alpine Linux sandbox** that runs real shell commands, **MCP server** support, and **persistent memory** that learns your project over time.
- **For your specific setup** — open-hardware project, Galaxy A15, heavy Termux use — the highest-value configuration is: install from F-Droid, use a **cloud API key (not the local Gemma model — your A15 is RAM/GPU constrained for it)**, enable the **Linux Sandbox** for in-chat scripting, wire up **MCP**

servers (DeepWiki, Context7, Fetch, Globalping), and seed the **soul/memory** with your project specifics. Treat Kai's sandbox as complementary to Termux, not a replacement.

- **Key caveats:** Kai depends on a non-free network service for cloud models (F-Droid flags this); on-device Gemma is marginal on the A15; and the app requests broad permissions (calendar, SMS, storage). Per a Factually long-term review, "real-world reliability varies: some users report slow or nonfunctional behavior and friction around API keys and permissions, so expect occasional instability and setup work for a smooth long-term experience."

Key Findings

1. **What it is.** Kai 9000 (package `com.inspiredandroid.kai`) is a Kotlin Multiplatform / Compose Multiplatform AI assistant. It's open source under Apache-2.0 at `github.com/SimonSchubert/Kai`, documented at `kai9000.com/docs`, Kai9000 and tagged "OpenClaw alternative in your pocket." On F-Droid the suggested version is 2.7.1 (build 108), added June 16, 2026, requiring Android 8.0+. The F-Droid build note states the version "is built and signed by the original developer, and guaranteed to correspond to this source tarball." Per AppBrain it

is "rated 4.58 out of 5 stars, based on 140 ratings" and has been "downloaded 6.7 thousand times," with frequent releases (the project is on a roughly weekly-to-fortnightly update cadence).

2. **The signature feature for you — the Linux Sandbox (Android only).** Kai bundles **proot** and downloads a ~3 MB Alpine Linux minirootfs to give the AI (and you) a real Linux userland: bash, coreutils, `apk` package manager, and one-tap install of curl, wget, git, jq, python3, pip, and Node.js. No root required. github There's an interactive terminal, a file browser, and the AI can run `execute_shell_command` (disabled by default — you must enable it). This is the bridge between conversational AI and the actual command-line work a hardware project involves.

3. **AI/LLM integrations are extensive.** The F-Droid and GitHub listings advertise "24 Services — One App, Every Model. Connect to ChatGPT, Gemini, Claude, DeepSeek, Llama, Mistral, Grok, and 17 more providers," while the live `multi-service` documentation enumerates 27 providers across three API formats (OpenAI-compatible, Gemini native, Anthropic native) plus on-device LiteRT: Atlas Cloud, OpenAI, Gemini, Anthropic, DeepSeek, Mistral, xAI, OpenRouter, Groq, NVIDIA, Cerebras, Ollama Cloud, LongCat, Together AI, Hugging Face, Venice, Moonshot, Z.AI (+ Coding Plan),

MiniMax, AIHubMix, Deep Infra, Fireworks, OpenCode, Public AI, the generic OpenAI-Compatible endpoint (defaults to `localhost:11434/v1` for Ollama/LM Studio), and a no-key **Free** tier. You configure a **fallback chain** — drag services to reorder; primary is tried first, then fallbacks, with Free as last resort.

4. **MCP (Model Context Protocol) support** lets Kai connect to remote tool servers via Streamable HTTP. It ships ~11 curated free, no-auth servers including **Fetch** (HTML→markdown), **DeepWiki** (AI docs for any GitHub repo), **Context7** (up-to-date library/framework docs), **Sequential Thinking**, **Globalping** (ping/traceroute/DNS from global probes), and CoinGecko. github DeepWiki and Context7 are directly useful for an open-hardware developer reading datasheets, library docs, and repos.

5. **Persistent memory + "soul."** Kai stores facts, preferences, learnings, and error-resolutions across conversations (four categories: General, Learning, Error, Preference). Memories reinforced 5+ times become "promotion candidates" that can be graduated into the permanent system prompt ("soul"). You define the soul via an editable system prompt with presets (Short & Direct, Tutor, Creative Writer) or your own text.

6. **Autonomous heartbeat + scheduled tasks.** A background "heartbeat" runs on a configurable interval (default 30 min, active hours 8:00–22:00) reviewing tasks, emails, and memories — staying silent unless something needs attention. Scheduled tasks support one-time (`execute_at`), recurring (cron), or `on_heartbeat` triggers. kai9000 On Android, a **Daemon Mode** foreground service keeps these running in the background.

7. **40+ tools and Interactive UI.** Tools include web search, fetch_url, calendar events, alarms, notifications, TTS, SSH host aliasing, and process management. The "Interactive UI" (kai-ui) lets the AI render tappable forms, tables, charts, and full app-like screens inside chat.

Details

Installation & first steps (your A15)

1. Install the **F-Droid client**, then install Kai 9000 from it (recommended over APK so you get update notifications; the F-Droid build is reproducible and developer-signed). It's also on Google Play.
2. Launch — you can chat immediately on the **Free tier** with no key.

3. For real work, open **Settings → Services → Add Service**, pick a provider, paste your API key (validated after an 800 ms debounce), pick a model, and drag to set priority. Add multiple services for fallback resilience.

AI model strategy: use cloud, not local, on the A15

This is the single most important configuration decision for your device. Kai *can* run Google Gemma E2B (~2.58 GB on disk) / E4B (~3.65 GB) on-device via LiteRT, but **your Galaxy A15 is a weak candidate**:

- Per GSMArena, the A15 5G uses a "Mediatek Dimensity 6100+ (6 nm): Octa-core (2x2.2 GHz Cortex-A76 & 6x2.0 GHz Cortex-A55); Mali-G57 MC2," and ships in "128GB 4GB RAM, 128GB 6GB RAM, 128GB 8GB RAM, 256GB 8GB RAM" variants (the 4G model uses the comparable Helio G99 with the same Mali-G57 GPU).
- Google advertises that the Gemma 3n family runs "with as little as 2GB (E2B) and 3GB (E4B) of memory," and its quantization (QAT) work "reduced the memory footprint of Gemma 4 E2B to 1GB." However, real-world peak working sets are higher (Google's own LiteRT model-card benchmarks on a flagship show E2B peak RSS in the ~2.7–3.4 GB range), and community crash reports indicate **4 GB devices are at or over the**

edge and prone to out-of-memory failures. 6 GB is marginal; 8 GB is the comfortable floor.

- Mid-range MediaTek GPUs frequently don't expose OpenCL to the app layer, so LiteRT tends to fall back to CPU. Expect roughly **low single-digit tokens/second on CPU** versus ~17–23 tok/s on a flagship — usable for a one-line query, painful for sustained chat.
- Sustained inference thermally throttles even flagships (decode can drop 30–50% within minutes); a passively-cooled budget phone will throttle faster.

Recommendation: Use a fast, cheap cloud provider as primary. Groq or Cerebras (very fast, generous free tiers), DeepSeek (cheap, strong reasoning), or Gemini (free tier, good tool use) are strong choices. Configure 2–3 in a fallback chain and leave Free enabled as last resort. Reserve local Gemma only for genuinely offline situations and only if you have the 8 GB A15 — and even then, prefer Qwen3 0.6B for quick validation (note: it's chat-only, can't call tools).

Linux Sandbox vs. Termux — how they relate

You already use Termux heavily, so the key insight is that **Kai's sandbox is a separate, app-private Alpine environment**, not your Termux environment. Kai's proot-based Alpine rootfs lives inside the app's private

storage; it cannot see your Termux `$HOME` or packages. Differences that matter:

- **Kai sandbox:** AI-driven. The LLM can write a Python script and run it, install `apk` packages, process data, and open generated files in Android apps via FileProvider. Each conversation gets a persistent bash session (cd/exports carry across calls within that conversation). Output limits ~15,000 chars/stream on Android; shell timeout default 30s, max 60s.
- **Termux:** Your hands-on, full-featured environment with a much larger package ecosystem (`pkg`), persistent storage, and `termux-api` hardware hooks (e.g., `termux-usb` for serial/USB work on hardware).

Workflow tip: Use Termux for your heavy lifting (flashing firmware, serial monitors, git, build toolchains) and use Kai's sandbox for AI-assisted micro-tasks: "write and run a script to parse this CSV of sensor readings," "compute the resistor divider values," "generate a self-contained HTML dashboard of this data and open it." Because Kai's `open_file` needs self-contained HTML (inline CSS/JS — FileProvider only grants the single file URI), explicitly ask it to inline everything when generating viewable output.

Bridging the two: Kai exposes `ssh_configure_host` (disabled by default). Since both apps run on the same phone, you can run an SSH/SFTP server in Termux (`pkg install openssh`, then `sshd`) and register it as an SSH host alias in Kai, letting Kai's sandbox reach into a Termux-hosted environment over `localhost`. Note SSH multiplexing (ControlMaster) is intentionally disabled — Android blocks the `link()` syscall OpenSSH uses for control sockets, so each `ssh` call does a full TCP + auth handshake. That's fine for occasional commands, slow for chatty loops.

MCP servers worth enabling for hardware work

Go to **Settings → Tools → Add MCP Server**. For an open-hardware project:

- **DeepWiki** — instant AI docs for any GitHub repo (great for understanding a library, an RTOS, or a board support package).
- **Context7** — current library/framework documentation (avoids stale training data when you're working with a specific Arduino/ESP-IDF/Zephyr API).
- **Fetch** — pull datasheets/spec pages into the chat as markdown.

- **Globalping** — network diagnostics if your hardware has a connectivity component.

All popular servers are free, require no API key, and auto-reconnect on app startup. MCP tools get a 60-second timeout (vs. 30s for native tools).

Memory & soul setup for your project

Seed Kai early. Write a soul like: *"You are an embedded-systems engineering assistant for [project name], an open-source hardware project using [MCU/board]. Prefer concise answers with code. Default to [language]. Cite datasheet sections when relevant."* Then let memory accumulate: pin facts (pinouts, toolchain versions, known-good configs) as memories. When a fix proves repeatedly useful (5+ reinforcements), promote it into the soul so it's always in context. Memory is injected into every system prompt (capped at 2,000 chars for local models, uncapped for cloud).

Automation: heartbeat & tasks

- Use **scheduled cron tasks** for recurring jobs: e.g., a daily task to summarize new commits or check a parts supplier. Tasks run via the full tool loop, so they can web-search, run shell commands, or call MCP servers. Cron uses the standard 5-field format (e.g., `0 9 * * *` = "Daily at 9:00").

- Enable **Daemon Mode** (Settings → General, Android only) if you want tasks/heartbeat to fire when the app is backgrounded. The daemon shows a persistent low-priority notification and, by design, requests no battery-optimization exemption — so on Samsung's One UI you should **manually exclude Kai from battery optimization** for reliable background execution. Kai re-asserts the daemon on every app foreground to recover from aggressive OEM process-killing.
- Pick a **cheaper/faster heartbeat model** override (Settings → heartbeat) so background checks don't burn your primary quota.

Configuration & data handling

- **Settings export/import:** Back up everything (services, soul, memory, tasks, MCP, tools, conversations) to a `kai-settings.json`, with selective Replace/Merge import. kai9000 Do this before experimenting. Note: API keys *are* included in the export; `encryption_key` and `daemon_enabled` are excluded.
- **Encryption:** On Android, API keys, passwords, and conversation history are stored in `EncryptedSharedPreferences` (AES-256-GCM values, keys via Android Keystore). All chat history is local; API calls go directly to your chosen

provider — Kai never proxies them (except the Free tier).

- **Tool safety guards:** 15-iteration tool-loop cap, repeated-call detection, 20,000-char result truncation, context-window overflow protection, and AI-powered history compaction at 70% of context. Good to know when a long sandbox session starts behaving oddly.
- **Permissions:** F-Droid lists broad permissions — foreground service, calendar read/write, shared storage, SMS read/send, phone state, alarms. Grant only what you use; the email/SMS/calendar tools are gated behind their feature toggles.

Reliability expectations

Independent reviews describe Kai as powerful but variable: some users report slowness or setup friction, and output quality varies because Kai routes to many different LLMs. It's best suited to technically competent, privacy-conscious users — which fits your profile. Stay on recent builds (the developer iterates quickly).

Recommendations

Stage 1 — Get running (15 min): Install from F-Droid. Add **Groq** or **Gemini** (free tiers, fast, good tool

support) as primary; add a second provider as fallback; leave Free enabled. Skip local Gemma.

Stage 2 — Enable your power tools: Turn on the **Linux Sandbox** (Settings → Linux Sandbox), install the basic package set (python3, git, jq, node). Enable `execute_shell_command`. Add MCP servers **DeepWiki + Context7 + Fetch**.

Stage 3 — Make it project-aware: Write a project-specific **soul**; manually store key project facts as memories. Set up one or two **cron tasks** for recurring chores. Enable **Daemon Mode** only if you need background automation, and exclude Kai from Samsung battery optimization.

Stage 4 — Bridge with Termux: Run `sshd` in Termux and register it via `ssh_configure_host` if you want Kai to drive your richer Termux environment; otherwise keep the two separate and use Kai's sandbox for AI micro-tasks only.

Thresholds that change the advice:

- If you upgrade to a phone with **8 GB+ RAM and a Snapdragon 8-series-class GPU**, on-device Gemma E4B becomes viable for offline privacy. (For reference, running E2B at full BF16 precision "requires approximately 8GB of VRAM, with additional memory needed for activations, context, and the KV cache" — the on-device

quantized builds are far smaller, but the 8 GB RAM bar is the realistic comfort threshold.)

- If your cloud quota becomes the bottleneck, add more providers to the fallback chain rather than switching to local.
- If background tasks silently stop firing, that's One UI killing the daemon — re-open Kai and check battery-optimization settings.

Caveats

- **Non-free network dependency:** F-Droid flags that Kai "promotes or depends entirely on a non-free network service" F-Droid for cloud models. Only the local Gemma path is fully free/offline, and that's marginal on the A15.
- **On-device inference is constrained on the A15** (RAM ceiling on 4 GB units, likely CPU-only fallback, thermal throttling). All A15-specific performance figures here are extrapolated from comparable mid-range Android hardware and Google's flagship benchmarks — no public benchmark tests the Helio G99 / Dimensity 6100+ with these models directly.
- **Sandbox ≠ Termux:** Kai's Alpine environment is app-private and isolated; it won't share your Termux setup, and by design it has no access to the Android host system.

- **Broad permissions** and community-reported reliability variance mean you should back up settings early and grant permissions conservatively.
- **Version/marketing drift:** Provider counts differ between channels (store/README say "24 services" while the live docs enumerate 27) and features change frequently; always confirm against the in-app Settings and the current `kai9000.com/docs`.